IdM Test Framework

Testing identity workflows across directories, databases and APIs

YAML scenarios and Java tests with shared connectors

Coworker introduction · 15 September 2026

Two ways to author a test

YAML

For consultants and test authors

Fixtures, ordered steps and assertions
Computed values and result captures

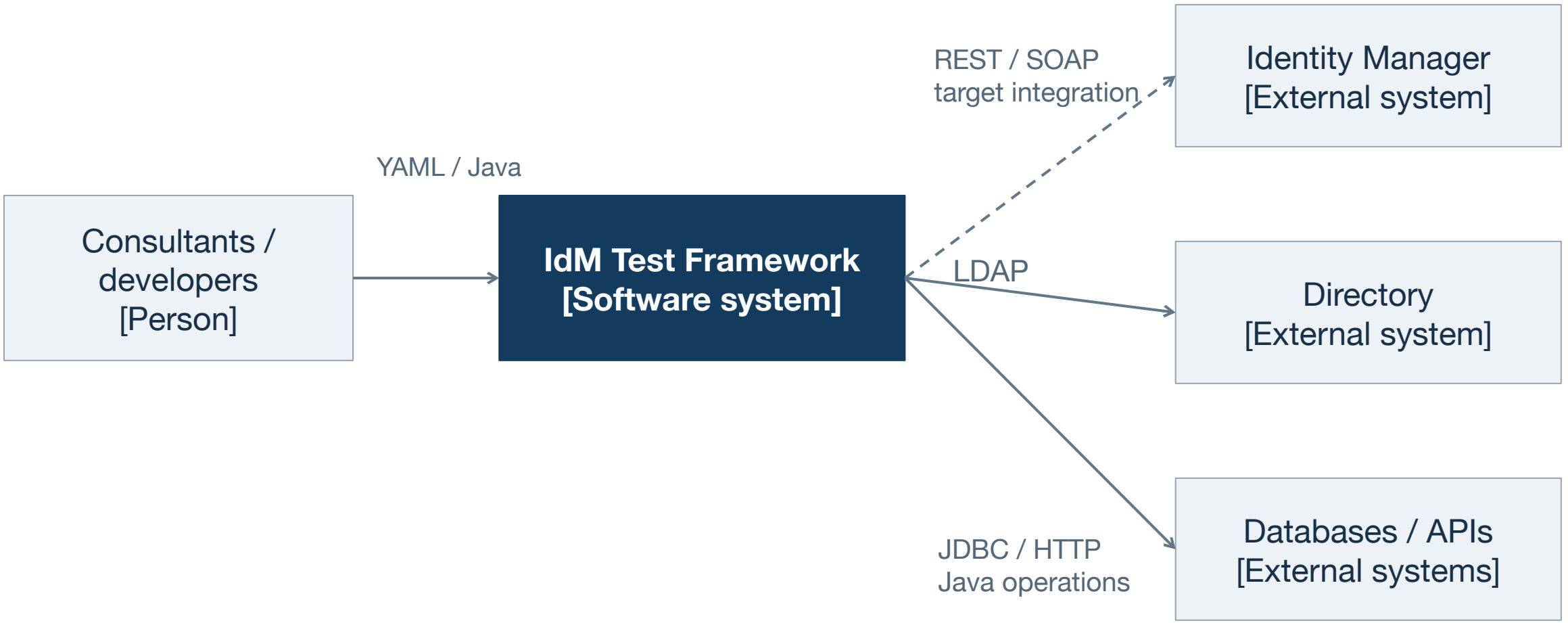
Java / JUnit

For developers

Direct connector calls
Broader operations and control flow

Both paths use the same connector implementations.

System context



Dashed: real IdM integration target; the local suite uses embedded fixtures.

A YAML scenario

Excerpt from the bundled workflow

```
- ldap.create:
    dn: "cn=${user.userId},ou=users"
    as: created
    # attributes omitted here

- ldap.verify:
    dn: "${created}"
    expect:
      givenName: ${user.firstName}
    wait: { timeout: 30s, poll: 2s }
```

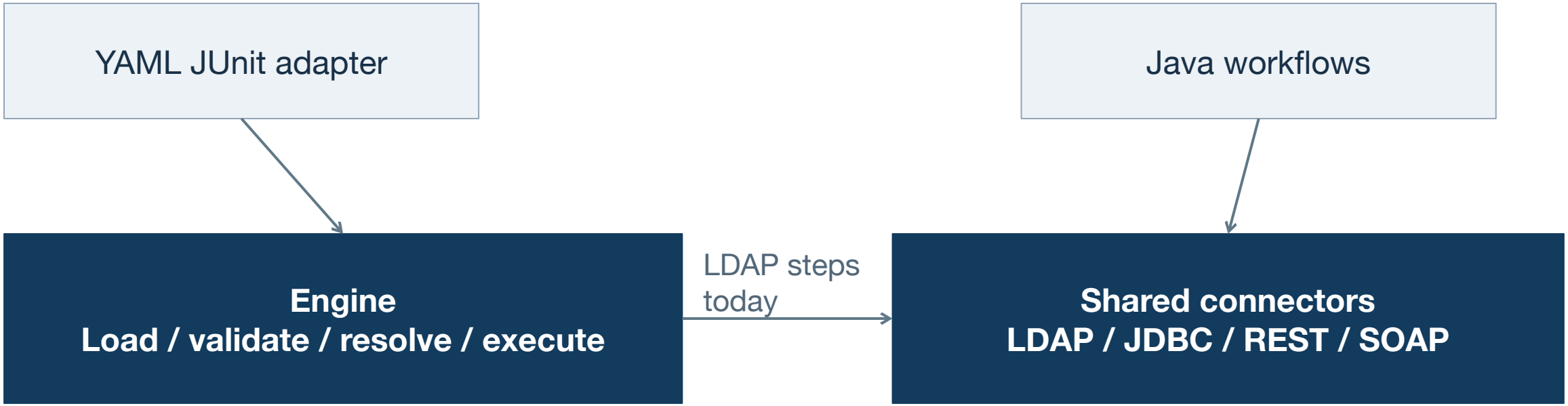
Create once

Capture the DN

Retry the assertion

Attempt cleanup

Architecture and shared connectors



Core and utilities: operation results, waiting and test data

Maven modules are code boundaries inside the test runtime.

Execution and failure handling



Only assertions retry. Values resolve once per step.

First failure stops later steps. Registered undo runs in reverse.
Cleanup is best effort; warnings remain in logs.

Local demonstration and results

Embedded LDAP + H2 + WireMock

320 tests passed in the 15 September baseline

```
./mvnw -pl idm-test-scenarios -am test \
  -Dtest=YamlScenarioTest \
  -Dsurefire.failIfNoSpecifiedTests=false
```

Outputs: scenario-results.json and Surefire reports

This demonstrates the framework, not real IdM provisioning.

Current scope and next steps

Available	YAML LDAP workflows, value expressions and captures Java LDAP, database, REST and SOAP APIs
Approved work	Additional YAML steps and scenario migration
Separate work	Real IdM wiring and validation CLI, UI and HTML reports are deferred

Reading path: overview → authoring guide → architecture guide